



Cryptography and Network Security

Integrated Situation - Technical Report

4202670024 GBOUSSOUBO IGNAZIANKI
ETT B-Tech, Year 4, ULK Polytechnic Institute

September 27, 2026

Abstract

This report documents the security assessment and toolkit developed for the ULK Polytechnic student records system, covering risk assessment, an encryption and integrity-checking programme, and network traffic filtering rules.

Contents

1	Risk Assessment	2
1.1	Assets, Vulnerabilities, and Consequences	2
1.2	Risk Ranking	2
1.3	Recommended Controls	2
2	Encryption and Integrity Toolkit	2
2.1	Functions	3
2.2	Test Evidence	3
3	Network Traffic Filtering	3
3.1	Environment Note	3
3.2	Network Assumptions	4
3.3	Firewall Rules	4
3.4	Test Results	4
4	Conclusion	4

1 Risk Assessment

1.1 Assets, Vulnerabilities, and Consequences

Asset		Vulnerability	Consequence
Student database	records	Weak staff passwords	Attacker guesses/brute-forces login, leading to data breach or record tampering
Inter-campus transfer	file	Unencrypted transfer	Data intercepted or altered in transit (man-in-the-middle)
Records server		Outdated software combined with guest network access	Unpatched server reachable by anyone on the guest network, leading to full compromise

Table 1: Assets, vulnerabilities, and consequences

1.2 Risk Ranking

Rank Risk		Likelihood	Impact	Reason
1	Outdated software + guest access	High	High	No credentials needed; matches the suspicious external connection attempts already observed
2	Weak staff passwords	Medium-High	High	Easy to exploit, gives full legitimate-looking access
3	Unencrypted file transfer	Medium	Medium-High	Attacker needs network-path position first

Table 2: Risk ranking by likelihood and impact

1.3 Recommended Controls

Risk	Control
Outdated software + guest access	Segment guest network away from the server; apply regular patching
Weak passwords	Enforce a strong password policy and multi-factor authentication (MFA)
Unencrypted transfer	Encrypt transfers (SFTP/TLS) and encrypt files before sending

Table 3: Recommended controls

2 Encryption and Integrity Toolkit

A Python command-line toolkit, `security_toolkit.py`, was developed using the `cryptography` library (Fernet symmetric encryption) and the built-in `hashlib` module (SHA-256).

2.1 Functions

- **Key generation:** creates a random Fernet key, stored outside the repository (`key.key`, excluded via `.gitignore`).
- **Encryption:** reads a student record file and writes an encrypted `.enc` version using the key.
- **Decryption:** reverses the process, reconstructing the exact original file from the `.enc` file and the key.
- **Integrity check:** computes a SHA-256 hash of a file and can later re-check the file against the saved hash to detect any modification.
- **Error handling:** every operation checks for missing files, missing keys, and invalid/wrong keys, and exits with a clear error message instead of crashing.

2.2 Test Evidence

The toolkit was run end-to-end against a sample student record file (`sample_student_record.csv`):

```
$ python3 security_toolkit.py encrypt sample_student_record.csv sample_student_record.
enc --key-file key.key
Encrypted 'sample_student_record.csv' -> 'sample_student_record.enc'

$ python3 security_toolkit.py decrypt sample_student_record.enc sample_student_record.
decrypted.csv --key-file key.key
Decrypted 'sample_student_record.enc' -> 'sample_student_record.decrypted.csv'

$ diff sample_student_record.csv sample_student_record.decrypted.csv
(no output -- files are identical)

$ python3 security_toolkit.py hash sample_student_record.csv --save
sample_student_record.sha256
SHA-256 of 'sample_student_record.csv': 55
e91847c70d981356b9c6f522e60c53c8e86e409598886389fad893f4dfd609
Saved to 'sample_student_record.sha256'

$ python3 security_toolkit.py verify sample_student_record.csv --hash-file
sample_student_record.sha256
OK: 'sample_student_record.csv' matches the recorded hash. No tampering detected.
```

This confirms: (1) encryption and decryption are correct and lossless, since the decrypted file is byte-for-byte identical to the original, and (2) the SHA-256 integrity check correctly identifies an unmodified file, and (as tested separately) correctly flags a modified one with a mismatch warning.

3 Network Traffic Filtering

3.1 Environment Note

No physical or virtual lab network was available for this exam. The rules below are the actual `iptables` commands that would be applied on the records server (or its gateway) in a real deployment. To still produce genuine, reproducible test evidence, a Python simulator (`firewall_simulator.py`) re-implements the identical rule logic, evaluated in the same top-to-bottom order as `iptables`, and runs real connection-attempt tests against it.

3.2 Network Assumptions

Item	Value
Records server	10.0.0.10
Service port	8443
Staff network (authorised)	10.0.1.0/24
Guest network (blocked)	10.0.2.0/24
Suspicious external address	203.0.113.50

Table 4: Network assumptions used for the firewall configuration

3.3 Firewall Rules

```
# (a) Block guest network access to the student records server
iptables -A INPUT -s 10.0.2.0/24 -d 10.0.0.10 -j DROP

# (b) Permit authorised staff network access to the service
iptables -A INPUT -s 10.0.1.0/24 -d 10.0.0.10 -p tcp --dport 8443 -j ACCEPT

# (c) Block other inbound access to that service
iptables -A INPUT -d 10.0.0.10 -p tcp --dport 8443 -j DROP

# Default policy
iptables -P INPUT DROP
```

3.4 Test Results

#	Test	Expected	Actual	Result
1	Staff → records service (10.0.1.50)	ACCEPT	ACCEPT	PASS
2	Guest → records service (10.0.2.75)	DROP	DROP	PASS
3	Unknown external host → service (203.0.113.50)	DROP	DROP	PASS

Table 5: Firewall test results (simulated)

All three tests passed, confirming the rules correctly permit staff traffic to the service while blocking guest and unrecognised external traffic.

4 Conclusion

This project identified and ranked three key risks to the student records system -weak staff passwords, unencrypted inter-campus file transfer, and an outdated, guest-accessible server and proposed practical controls for each. A Python toolkit was implemented and tested to provide file encryption, decryption, and SHA-256 integrity verification, successfully demonstrating correct round-trip encryption and tamper detection. Firewall rules were designed to restrict access to the records server to authorised staff only, and validated through a rule-equivalent simulation, with all three required tests passing.

GitHub repository: <https://github.com/Itzspencer235/cryptography-network-security-exam>